EXPERIMENTS — break OneShot yourself

Each experiment gives the exact commands, what you should see, and what it teaches. Every one of them was run while building the project. The "you should see" sections describe the outcomes that were observed (IDs, PIDs and timings will differ on your machine).

Setup (once): `uv sync --extra docs`. Experiments 3 onwards use the default ports (OneShot API `8700`, provider `8701`) and keep their state in `./var`. Run `rm -rf var` between experiments to start fresh. Stop background processes with `pkill -f fakeprovider; pkill -f "oneshot worker"`, or Ctrl-C in their terminals.

---

1. Run every failure scenario

```
uv run oneshot demo
```

You should see: 12 scenarios, each printing OneShot's event timeline, the provider's own log, the provider's ledger and `[PASS]` checks, and finally a scorecard with every row `PASS` (about 25 s).

Teaches: the whole lifecycle at a glance; every claim is checked against the provider's independent ledger, not against OneShot's beliefs.

2. Run one scenario and keep its files

```
uv run oneshot demo --list
uv run oneshot demo worker-crash --keep
sqlite3 <printed dir>/worker-crash/oneshot.db "select number, kind, outcome, detail from attempts"
```

You should see: the `worker-crash` story, then attempt 1 with outcome `CRASHED` and attempt 2 with `APPLIED`. In `provider.log`, the second request is answered with `REPLAY of stored response`.

Teaches: the database *is* the story; you can audit any action after the fact.

3. Run the stack by hand and use the API

```
make dev        # provider :8701, API :8700, one worker. Open http://127.0.0.1:8700/docs
# in another terminal:
curl -i -X POST localhost:8700/actions -H 'Idempotency-Key: order-1' -H 'Content-Type: application/json'
\
     -d '{"type":"payment.charge","payload":{"amount_cents":1500,"customer":"cus_1"}}'
curl -i -X POST localhost:8700/actions -H 'Idempotency-Key: order-1' -H 'Content-Type: application/json'
\
     -d '{"type":"payment.charge","payload":{"amount_cents":1500,"customer":"cus_1"}}'
curl -i -X POST localhost:8700/actions -H 'Idempotency-Key: order-1' -H 'Content-Type: application/json'
\
     -d '{"type":"payment.charge","payload":{"amount_cents":9999,"customer":"cus_1"}}'
curl -s localhost:8700/actions
```

You should see: `201 Created`, then `200 OK` with `idempotent-replayed: true` and the same id, then `422` for the changed payload. Omitting the header gives `400`. The action list shows one action, `SUCCEEDED`, with a `provider_effect_id`.

Teaches: layer-1 duplicate protection (doc 03): the agent's key identifies an *intent*.

4. Lose a response on purpose

With `make dev` running:

```
curl -X POST localhost:8701/_admin/faults -d '{"mode":"lost_response"}'
curl -s -X POST localhost:8700/actions -H 'Idempotency-Key: order-2' -H 'Content-Type: application/json'
\
     -d '{"type":"payment.charge","payload":{"amount_cents":2500,"customer":"cus_2"}}'
uv run oneshot show <id from the response>
```

You should see: `IN_FLIGHT -> PENDING connection dropped before a response arrived; outcome unknown -> retry … with the SAME idempotency key`, then `SUCCEEDED … replayed original result`. The provider log shows `APPLIED …, response deliberately dropped` followed by `REPLAY of stored response (no new effect)`.

Teaches: an ambiguous outcome plus provider-side idempotency gives one charge, without OneShot ever knowing whether the first attempt worked.

5. `kill -9` the worker while its request is in the air

```
rm -rf var; mkdir -p var
uv run fakeprovider --port 8701 --db var/provider.db &
uv run oneshot worker 2> worker-a.log &
curl -X POST localhost:8701/_admin/faults -d '{"mode":"slow_success","delay_seconds":8}'
uv run oneshot submit payment.charge '{"amount_cents": 3500, "customer": "cus_3"}' --key order-3
sleep 1
kill -9 <PID>      # the number in "[worker worker-<PID>] started" inside worker-a.log
uv run oneshot list                # -> IN_FLIGHT
uv run oneshot worker --drain      # waits for the 15 s lease to expire, then recovers
```

> Kill the PID from the worker's id. `pgrep -f "oneshot worker"` may return the `uv run` wrapper instead, and killing that leaves the real worker running. This happened while the experiment was being written.

You should see: after about 15 s, `send #1: CRASHED -> PENDING: recovered: worker worker-<PID> was lost during attempt 1 …`, then `send #2: APPLIED -> SUCCEEDED: … replayed original result`. `curl localhost:8701/v1/effects` lists exactly **one** effect.

Teaches: write-ahead intent + leases + same-key retry = crash recovery without a duplicate (doc 04). Try `ONESHOT_LEASE_SECONDS=5` to make recovery faster, and notice why it must stay above twice `ONESHOT_PROVIDER_TIMEOUT`.

6. Make a duplicate happen: provider slower than the settle window

```
rm -rf var; mkdir -p var
uv run fakeprovider --port 8701 --db var/provider.db &
curl -X POST localhost:8701/_admin/faults -d
'{"mode":"slow_success","endpoint":"/v1/messages","delay_seconds":6}'
uv run oneshot submit message.send '{"to":"+15550100","body":"refund sent"}' --key notify-1
ONESHOT_SETTLE_SECONDS=1 uv run oneshot worker --drain
sleep 5; curl -s localhost:8701/v1/effects; uv run oneshot reconcile; echo "exit=$?"
```

You should see: `AMBIGUOUS -> VERIFYING`, `verify #1: ABSENT -> PENDING: no record 1s after the last send -> treated as NOT applied`, `send #2: APPLIED -> SUCCEEDED`. A few seconds later the provider holds **two** effects for the action, reconciliation reports `DUPLICATE_EFFECT`, and the command exits `1`.

Teaches: verify-then-retry rests on an assumption (the provider settles in time). Break the assumption and you get the duplicate. Reconciliation detects it; it cannot undo it. Re-run with `ONESHOT_SETTLE_SECONDS=10` and only one message is sent.

7. Misconfigure the retention window (and then fix it)

```
# 7a: the provider forgets keys after 2 s, but OneShot believes 24 h (the default)
rm -rf var; mkdir -p var
uv run fakeprovider --port 8701 --db var/provider.db --idempotency-ttl 2 &
curl -X POST localhost:8701/_admin/faults -d '{"mode":"lost_response"}'
uv run oneshot submit payment.charge '{"amount_cents":1500,"customer":"cus_1"}' --key order-7
ONESHOT_BACKOFF_BASE=4 ONESHOT_BACKOFF_JITTER=0 uv run oneshot worker --drain
curl -s localhost:8701/v1/effects        # -> TWO effects

# 7b: tell OneShot the truth
pkill -f fakeprovider; rm -rf var; mkdir -p var
uv run fakeprovider --port 8701 --db var/provider.db --idempotency-ttl 2 &
curl -X POST localhost:8701/_admin/faults -d '{"mode":"lost_response"}'
uv run oneshot submit payment.charge '{"amount_cents":1500,"customer":"cus_1"}' --key order-7
ONESHOT_BACKOFF_BASE=4 ONESHOT_BACKOFF_JITTER=0 ONESHOT_KEY_RETENTION=2 ONESHOT_KEY_MARGIN=0.5 \
  ONESHOT_SETTLE_SECONDS=1 uv run oneshot worker --drain
curl -s localhost:8701/v1/effects        # -> ONE effect
```

You should see: in 7a, a same-key retry 4 s later, which the provider no longer recognises, so there are 2 effects. In 7b, `AMBIGUOUS -> VERIFYING: … retry with same key no longer safe` followed by `verify #1: FOUND -> SUCCEEDED`, and 1 effect.

Teaches: idempotency keys protect only inside the provider's retention window, and OneShot must be told that window truthfully (doc 02, G3 in the overview).

8. Provider completely down: a safe FAILED

```
pkill -f fakeprovider; rm -rf var; mkdir -p var
uv run oneshot submit payment.charge '{"amount_cents":1500,"customer":"cus_1"}' --key order-8
ONESHOT_BACKOFF_BASE=0.1 uv run oneshot worker --drain
uv run oneshot list --json
```

You should see: five `RETRYABLE` attempts (`could not connect (ConnectError); request never sent`) and then `FAILED` with `maybe_applied: false`.

Teaches: connection refused proves the request never left, so FAILED is a *true* statement here. Compare with the demo's `provider-crash-unknown`, where a crash *after* sending leads to UNKNOWN instead.

9. Wipe OneShot's database; the evidence survives

```
rm -rf var; mkdir -p var
uv run fakeprovider --port 8701 --db var/provider.db &
uv run oneshot submit payment.charge '{"amount_cents":1500,"customer":"cus_9"}' --key order-9
uv run oneshot worker --drain
rm -f var/oneshot.db var/oneshot.db-wal var/oneshot.db-shm
uv run oneshot reconcile; echo "exit=$?"
```

You should see: `Reconciliation (applied): 0 actions vs 1 provider effects` with one `ORPHAN_EFFECT` row, and `exit=1`.

Teaches: the provider's ledger is independent evidence; OneShot losing its memory does not erase what happened (doc 05).

10. Two workers, one queue

```
rm -rf var; mkdir -p var
uv run fakeprovider --port 8701 --db var/provider.db &
for i in $(seq 1 20); do uv run oneshot submit payment.charge "{\"amount_cents\": $((100+i)),
\"customer\": \"cus_$i\"}" --key bulk-$i; done
uv run oneshot worker --drain --id A 2> a.log & uv run oneshot worker --drain --id B 2> b.log
sleep 2; grep -c 'send #' a.log b.log; uv run oneshot reconcile
```

You should see: the work split between A and B (one observed run: 9 and 11 sends), 20 actions with 20 sends in total, every action `MATCH` in reconciliation, and no duplicates.

Teaches: the atomic claim (`BEGIN IMMEDIATE`) gives each action to exactly one worker.

11. Ask for an unsafe configuration

```
ONESHOT_PROVIDER_TIMEOUT=5 ONESHOT_LEASE_SECONDS=6 uv run oneshot list
```

You should see: `invalid configuration: lease_seconds must be more than twice provider_timeout_seconds` (exit code 2).

Teaches: some settings are safety properties, so OneShot refuses to start rather than run unsafely.

12. Break a guarantee and watch the tests catch it

```
# in oneshot/action_types.py, set MESSAGE_SEND's idempotent=False to idempotent=True (a lie)
uv run pytest -q tests/test_invariants.py -k message
git checkout oneshot/action_types.py
```

You should see: failures in `lost_response-message.send` , `slow_success-message.send` and `crash_after_apply-message.send` : the provider ignores the key, so blind retries duplicate.

Teaches: capability flags must describe reality; the invariant suite is the safety net for every future change (doc 07).

13. Add your own failure mode

Follow the worked example in `docs/src/07-how-to-extend.md` . For instance, add a `partial_timeout` that applies the effect after 0.5 s *and* drops the connection. Run `uv run pytest -q tests/test_invariants.py` before writing any scenario-specific test, and see which guarantees hold automatically.